

INDEX

Name: Prinkal Shahi Class: 4th Sem
 Section: D Roll No.: 28F24C5223 Subject: OS-lab

Sl. No.	Date	Title	Page No.	Teacher's Sign. / Remarks
1.	1	LAB - 1	1-8	dy 13/3
2.	8	FCFS	9 -	dy 13/3
3.	06-3	SJF (Preemptive & Non-preemptive)	11	dy
4.	27-03	Priority (Pre-emptive & Non-preemptive)	12	
5.	27-03	Round-Robin & Multilevel with Preemptive	17	
6.	10-04	Rate Monotonic	-25	dy 13/3
		Earliest-deadline first		
		Proportional Scheduling		
7.	24-04	Bounded Buffer & Dining Philosopher	26-30	dy 13/3
8.	08-05	Banker's algorithm	31-37	dy 13/3
		Deadlock Detection	38-43	dy 13/3
9.	15-05	Memory allocation	10	dy 13/3
		Best-fit		
		First-fit		
		Worst-fit		
10.	22-05	Page Replacement Algorithms (FIFO), (LRU), (Optimal)		

O/P

Enter n for size: 2
Enter element [1] [1] = 2
" [1] [2] = 7
" [2] [1] = 3
" [2] [2] = 6
Left diagonal sum = 5

③ WAP to find the sum & rows & columns of a matrix.

```
#include <stdio.h>

int main() {
    int m, n; int i, j;
    printf("Enter size m, n");
    scanf("%d %d", &m, &n);
    int mat[m][n];
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &mat[i][j]);
        }
    }
    for (int i = 0; i < m; i++) {
        int rsum = 0;
        for (int j = 0; j < n; j++) {
            rsum = rsum + mat[i][j];
        }
        printf("Row sum %d\n", rsum);
    }
    for (int i = 0; i < n; i++) {
        int csum = 0;
        for (int j = 0; j < m; j++) {
            csum = csum + mat[j][i];
        }
        printf("Column %d sum %d\n", i+1, csum);
    }
}
```


2

```

else if (arr[i] < small && arr[i] != small) {
    small = arr[i];
}
}

```

O/P

Enter no. of elements in array: 5
 Enter elements:
 5 9 8 6 5
 second smallest element: 5

② WAP to find sum of left diagonal of a matrix.

→ #include <stdio.h>

```
int main() {
```

```
    int n;
```

```
    printf("Enter n for size");
```

```
    scanf("%d", &n);
```

```
    int mat[n][n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            printf("Enter element [%d][%d]: ", i, j);
```

```
            scanf("%d", &mat[i][j]);
```

```
        }
```

```
    }
```

```
    int sum = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (i == j) sum += mat[i][j];
```

```
        }
```

```
    }
```

```
}
```

```
    printf("left diagonal sum = %d", sum);
```

Program

① WAP to find the second smallest element in an array.

```
→ #include <stdio.h>
int main() {
    int n;
    printf("Enter no. of elements in an array");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the elements");
    for (int i=0; i<n; i++) {
        scanf("%d", &arr[i]);
    }
    if (n < 2) {
        printf("Should have at least 2 elements\n");
        return 0;
    }
    int small, small;
    if (arr[0] < arr[1]) {
        small = arr[0];
        small = arr[1];
    } else {
        small = arr[1];
        small = arr[0];
    }
    for (int i=0; i<n; i++) {
        if (arr[i] < small) {
            small = small;
            small = arr[i];
        }
    }
```



```

    } else if (arr[i] > slar && arr[i] >
        lar) {
        slar = arr[i];
    }
}
print f ("second largest element: %d\n", slar);
}

```

O/P

Enter size: 5

Enter elements: 2 3 4 6 9

second largest element is 6.

```
printf("Total no. of duplicates: %d\n", c);
}
```

O/P

Enter no. of elements: 5

2 6 3 1 1

Total no. of duplicates: 2

⑤ WAP to find second largest element in array.

→ #include <stdio.h>

int main()

{ int n;

printf("Enter size: \n");

scanf("%d", &n);

int arr[n];

printf("Enter elements:");

for (int i=0; i<n; i++)

scanf("%d", &arr[i]);

}

if (n < 2)

printf("Array must have > 2 elements\n");

return 0;

{ int lar, slar;

if (arr[0] > arr[1])

lar = arr[0];

slar = arr[1];

{ else

lar = arr[1];

slar = arr[0];

{ for (int i=0; i<n; i++)

if (arr[i] > lar)

slar = lar;

lar = arr[i];

c/p Enter size - 2, 2
 Enter elements - 3
 6
 9
 4

Row 1 sum = 9

Row 2 sum = 13

Column 1 sum = 12

Column 2 sum = 10

④ WAP to calculate total number of duplicate element in array.

```
→ #include <stdio.h>
int main() {
    int n;
    printf("Enter size:");
    scanf("%d", &n);
    int a[n];
    for (int i=0; i<n; i++) {
        scanf("%d", &a[i]);
    }
    int c = 0;
    int v[n];
    for (int i=0; i<n; i++)
        v[i] = 0;
    for (int i=0; i<n; i++) {
        if (v[i] == 1) continue;
        int d=0;
        for (int j=i+1; j<n; j++) {
            if (a[i] == a[j]) {
                d=1;
                v[j]=1;
            }
        }
    }
}
```

LAB-1

Q1) Write a program to simulate CPU scheduling algorithm.

- FCFS (First Come first serve).
- SJF

→ #include <stdio.h>
int main()

{
int n, i;

printf("Enter number of process:");

scanf("%d", &n);

int at[10], bt[10], ct[10], tat[10];

int wt[10], rt[10];

for (i=0; i<n; i++)

{
printf("Enter at for P: %d", i+1);

scanf("%d", &bt[i]);

}

st[0] = at[0];

ct[0] = st[0] + bt[0];

for (i=1; i<n; i++)

{
if (ct[i-1] < at[i])

st[i] = at[i];

else

st[i] = ct[i-1];

ct[i] = st[i] + bt[i];

for (i=0; i<n; i++)

{
tat[i] = ct[i] - at[i];

wt[i] = tat[i] - bt[i];

rt[i] = st[i] - at[i];

}

printf("\n P | AT | RT | ST | CT |
AT | WT | RT \n");

for (i=0; i<n; i++)

{
printf("%d\t%d\t%d\t%d\t%d\t",

Enter position to delete: 7
Invalid pos

O/P

Enter no. of elements: 5
Enter elements:

1

2

3

4

5

Enter position to delete: 2
Array after deletion

10/3/26

1 3 4 5

6. Write a program in C to delete an element at a desired position from an array.

```
#include <stdio.h>
int main() {
    int n, i, pos;
    printf("Enter number of elements:");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        printf("Enter position to delete:");
        scanf("%d", &pos);
    }
    if (pos < 1 || pos > n)
        printf("Invalid pos");
    return 0;

    for (i = pos - 1; i < n - 1; i++)
        arr[i] = arr[i + 1];

    n--;
    printf("Array after deletion:\n");
    for (i = 0; i < n; i++)
        printf("%d", arr[i]);
    return 0;
}
```

O/P Enter number of elements: 5

Enter elements: 4

\$ 1

\$ 2

\$ 3

\$ 4

~~Total duplicate: 2~~

5

- ③ Write a C program to simulate the CPU scheduling algorithm to find turn around time and waiting time.
 → Priority (pre-emptive & Non-pre-emptive)
 1) Preemptive

```
#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0;
    int bt[10], pr[10], wt[10], tat[10],
        rt[10];
    int smallest, finish_time;
    int pid[10];
    int min = 9999;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("In Process %d\n", i+1);
        pid[i] = i+1;
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &pr[i]);
        rt[i] = bt[i];
    }

    while (completed != n) {
        smallest = 9;
        for (i = 0; i < n; i++) {
            if (pr[i] < pr[smallest] ||
                (pr[i] == pr[smallest] &
                 rt[i] > 0)) {
                smallest = i;
            }
        }
        rt[smallest]--;
    }
}
```

OP-2

Enter number of process: 5

Enter AT for P1: 0

Enter BT for P1: 10

Enter AT for P2: 3

Enter BT for P2: 5

Enter AT for P3: 5

Enter BT for P3: 2

Enter AT for P4: 6

Enter AT for P4: 6

Enter AT for P5: 8

Enter BT for P5: 4

1 SJF Non-preemptive

2 SJF preemptive

Enter choice: 2

P	AT	BT	ST	CT	AT	TAT	WT	RT
P1	0	10	0	27		27	17	0
P2	3	5	3	10		7	2	0
P3	5	2	5	7		2	4	0
P4	6	6	14	20		14	8	8
P5	8	4	10	14		6	2	2

13/3/20


```

        }
        b+ [i], st[i], ct[i], tat[i],
        wt[i], rt[i]);
    }
    return 0;
}

```

O/P - 1

Enter number of process: 5
 Enter AT for P1: 0
 Enter BT for P1: 10
 Enter AT for P2: 3
 Enter BT for P2: 5
 Enter AT for P3: 5
 Enter BT for P3: 2
 Enter AT for P4: 6
 Enter BT for P4: 6
 Enter AT for P5: 8
 Enter BT for P5: 4

1. SJF non preemptive
 2. SJF preemptive
 Enter your choice: 1

P	AT	RT	ST	CT	TAT	WT	RT
P1	0	10	0	10	10	0	0
P2	3	5	16	21	28	13	13
P3	5	2	10	12	7	5	5
P4	0	6	21	27	21	15	15
P5	8	4	12	16	8	4	4

Page 15

```

p[i].pid = i + 1;
printf("Burst Time");
scanf("%d", &p[i].burst_time);
printf("Priority");
scanf("%d", &p[i].priority);
}
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        if (p[i].priority > p[j].priority) {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

```

```

p[0].waiting_time = 0;
for (i = 1; i < n; i++) {
    p[i].waiting_time = p[i-1].waiting_time +
        p[i-1].burst_time;
}
for (i = 0; i < n; i++) {
    p[i].turnaround_time = p[i].waiting_time + p[i].burst_time;
}
printf("\nPID\tBurst Time\tPriority\tWaiting Time\tTurnaround Time\n");
float total_cot = 0, total_tat = 0;
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].burst_time,
        p[i].priority,
        p[i].waiting_time,
        p[i].turnaround_time);
}

```


Burst Time: 5

Priority: 3

Process 5

Burst Time: 2

Priority: 1

PID	BT	PR	WT	TAT
1	2	1	0	2
2	3	2	4	7
3	4	2	7	11
4	5	3	11	16
5	2	1	2	4

ii) Non-preemptive

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, j;
```

```
    int bt[20], pr[20], wt[20], tat[20];
```

```
    struct process {
```

```
        int pid;
```

```
        int burst time;
```

```
        int priority;
```

```
        int waiting time;
```

```
        int turn around time;
```

```
    };
```

```
    int main () {
```

```
        int n, i, j;
```

```
        struct process p[50], temp;
```

```
        printf("Enter number of processes: ");
```

```
        scanf("%d", &n);
```

```
        for (i=0; i<n; i++) {
```

```
            printf("\nEnter details for process %d\n", i+1);
```

Date:
 Page: 19

```

if (wt[smallest] == 0) {
    completed++;
    finish_time = time + 1;
    wt[smallest] = finish_time -
        bt[smallest];
    if (wt[smallest] < 0)
        wt[smallest] = 0;
}
time++;
for (i=0; i<n; i++) {
    tat[i] = bt[i] + wt[i];
}
printf("\n PID \t BT \t PR \t WT \t TAT \n");
for (i=0; i<n; i++) {
    printf("%d \t %d \t %d \t %d \t %d \n",
        pid[i], bt[i], pr[i], wt[i], tat[i]);
}
return 0;
}

```

O/P Enter number of processes: 5

Process 1

Burst Time: 2

Priority: 1

Process 2

Burst Time: 3

Priority: 2

Process 3

Burst Time: 4

Priority: 2

Process 4


```

print f("Enter time quantum:");
scanf ("%d", & quantum);
int time = 0, done;
do {

```

```

    done = 1;
    for (int i = 0; i < n; i++) {
        if (p[i].remaining_time > 0 &&
            p[i].arrival_time <= time) {
            done = 0;
            if (p[i].remaining_time > quantum) {
                time += quantum;
                p[i].remaining_time -= quantum;
            } else {
                time = p[i].remaining_time;
                p[i].completion_time = time;
                p[i].turnaround_time = p[i].
                    completion_time - p[i].arrival_time;
                p[i].waiting_time = p[i].
                    turnaround_time - p[i].burst_time;
                p[i].remaining_time = 0;
            }
        }
    }
}

```

```

    int all_waiting = 1;
    for (int i = 0; i < n; i++) {
        if (p[i].remaining_time > 0 && p[i].arrival_time
            <= time) {
            all_waiting = 0;
            break;
        }
    }
}

```

```

    if (all_waiting) time++;
}

```

LAB-3 SIMULATION OF MULTI-LEVEL Queue Scheduling

Round Robin

```
#include <stdio.h>
struct process {
    int id;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int completion_time;
    int waiting_time;
    int turnaround_time;
};

int main() {
    int n, quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Enter arrival Time for process %d",
            p[i].id);
        scanf("%d", &p[i].arrival_time);
        printf("Enter burst Time for process %d",
            p[i].id);
        scanf("%d", &p[i].burst_time);
        p[i].remaining_time = p[i].burst_time;
        p[i].completion_time = 0;
        p[i].waiting_time = 0;
        p[i].turnaround_time = 0;
    }
}
```



```

total_wt += p[i].waiting_time;
total_tat += p[i].turnaround_time;
}
printf("\n Average Waiting Time = %.2f",
total_wt/n);
printf("\n Average Turnaround Time = %.2f",
total_tat/n);
return 0;
}

```

Output Enter number of processes: 3
 Enter details for Process 1
 Burst Time: 10
 Priority: 2
 Enter details for Process 2
 Burst Time: 5
 Priority: 1
 Enter details for Process 3
 Burst Time: 8
 Priority: 3

PID	Burst Time	Priority	Waiting Time	Turnaround Time
PID	BT	PR	WT	TAT
2	5	1	0	5
1	10	2	5	15
3	8	3	15	23

Date _____
Page 21

```

p_list[i].id = i + 1;
printf("Enter arrival time for P./d: ", p_list[i].id);
scanf("%d", &p_list[i].arrival_time);
printf("Enter burst time for P./d", p_list[i].id);
scanf("%d", &p_list[i].burst_time);
printf("Enter type (system/user) for P./d:", p_list[i].id);
scanf("%s", p_list[i].type);
p_list[i].remaining_time = p_list[i].burst_time;
p_list[i].completion_time = 0;
}

```

```

for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (p_list[i].arrival_time > p_list[j].arrival_time) {
            temp = p_list[i];
            p_list[i] = p_list[j];
            p_list[j] = temp;
        }
    }
}

```

```

}

struct Queue systemQ = {0, -1, 0};
struct Queue userQ = {0, -1, 0};
int current_time = 0, completed = 0, i = 0;
struct Process *current_process = NULL;
while (completed < n) {
    while (i < n && p_list[i].arrival_time == current_time) {
        if (strcmp(p_list[i].type, "system") == 0)
            enqueue(&systemQ, &p_list[i]);
        else
            enqueue(&userQ, &p_list[i]);
        i++;
    }
}

```


Multilevel with Preemptive

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Process {
```

```
    int id;
```

```
    int arrival_time;
```

```
    int burst_time;
```

```
    int remaining_time;
```

```
    int completion_time;
```

```
    char type[10];
```

```
};
```

```
struct Queue {
```

```
    int front, rear, size;
```

```
    struct Process *arr[100];
```

```
};
```

```
void enqueue (struct Queue *q, struct Process *p)
```

```
{  
    q->arr[++q->rear] = p;
```

```
    q->size++;
```

```
}
```

```
struct Process *dequeue (struct Queue *q)
```

```
{  
    if (q->size == 0) return NULL;
```

```
    struct Process *p = q->arr[q->front++];
```

```
    q->size--;
```

```
    return p;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process plist[n];
```

```
    for (int i=0; i<n; i++) {
```

```

printf("\n Process \t AT \t BT \t CT \t TAT \t WT \n");
int total_wait = 0, total_turn = 0;
for (int i = 0; i < n; i++) {
    printf("P%d \t /d \t /d \t /d \t /d \t /d \n",
           p[i].id, p[i].arrival_time,
           p[i].burst_time, p[i].completion_time,
           p[i].turnaround_time, p[i].waiting_time);
    total_wait += p[i].waiting_time;
    total_turn += p[i].turnaround_time;
}
printf("\n Average Waiting Time: %.2f",
       (float) total_wait / n);
printf("\n Average Turnaround Time: %.2f",
       (float) total_turn / n);
return 0;
}

```

Output

Enter number of processes: 3
 Enter arrival time for process 1: 0
 Enter burst time for process 1: 5
 Enter arrival time for process 2: 1
 Enter burst time for process 2: 3
 Enter arrival time for process 3: 2
 Enter burst time for process 3: 1
 Enter time for quantum: 2

Process	AT	BT	CT	TAT	WT
P1	0	5	9	9	4
P2	1	3	8	7	4
P3	2	1	5	3	2

Average waiting Time: 3.33
 Average Turnaround Time: 6.33

IP	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
1	User	2	8	12	7	10
2	System	1	4	5	4	8
3	User	3	2	14	9	11

End

Average waiting Time: 5.00

Average Turnaround Time: 8.50

Average Response Time: 7.83

Comparison for Round Robin

when observed with two process identical arrival & burst time, but different time quantum (one higher & one lower) low one gives less response time but increase no. of context switching & vice versa for long quantum.

Ques 3

O/P - 2

Enter number of processes: 6

Enter AT for P1: 5

Enter BT for P1: 5

Enter AT for P2: 4

Enter BT for P2: 6

Enter AT for P3: 6

Enter BT for P3: 7

Enter AT for P4: 11

Enter BT for P4: 9

Enter AT for P5: 2

Enter BT for P5: 2

Date _____
Page 28

```

int wt = tat - plist[k].burst_time;
printf("P/d\t/d\t\t/d\t/d\t\t/d\t\t/d\t\t/d\t\t/s\n", plist[k].id,
plist[k].arrival_time, plist[k].burst_time, plist[k].completion_time, tat,
wt, plist[k].type);
}
return 0;
}

```

O/P Enter no. of processes: 4
 Process 0
 Enter arrival time: 0
 Enter burst time: 5
 Enter type (0=System, 1=User): 0

Process 1
 Enter arrival time: 2
 Enter burst time: 3
 Enter type (0=System, 1=user) = 1

Process 2
 Enter arrival time: 1
 Enter burst time: 4
 Enter type (0=System, 1=user) = 0

Process 3
 Enter arrival time: 3
 Enter burst time: 2
 Enter type (0=System, 1=user) = 1

Enter time quantum: 4


```

if (current_process != NULL) {
    if (strcmp(current_process->type,
        "user") == 0 && systemB.size > 0) {
        enqueue(&userQ, current_process);
        current_process = NULL;
    }
}

if (current_process == NULL) {
    if (systemQ.size > 0)
        current_process = dequeue(&systemQ);
    else if (userQ.size > 0)
        current_process = dequeue(&userQ);
    else {
        current_time++;
        continue;
    }
}

current_process->remaining_time--;
current_time++;
if (current_process->remaining_time == 0) {
    current_process->completion_time = current_time;
    printf("Process %d (%s) completed at\n", current_process->id,
        current_process->type, current_time);
    completed++;
    current_process = NULL;
}

printf("In Process \tAT \tBT \tCT \tTAT \tWT\n",
    current_process->type);
for (int k=0; k<n; k++) {
    int tat = plist[k].completion_time -
        plist[k].arrival_time;
}

```

```

}
else {
    time += t;
}
}
}
printf ("ID | BF | Deadline | CT | WT | TAT | n");
for (int i=0; i<n; i++) {
    printf ("%d | %d | %d | %d | %d | %d | n",
            t[i].id, t[i].execution, t[i].deadline, ct[i], wt[i], t[i].TAT);
}
}

```

```

void RMS (Task t[], int n) {
    printf ("\n Rate monotonic Scheduling (RMS) \n");
    int time=0, completed=0;
    int remaining [MAX], ct [MAX], wt [MAX], tat [MAX];
    float utilization=0;
    for (int i=0; i<n; i++) {
        remaining[i] = t[i].execution;
        utilization += (float) t[i].execution / t[i].period;
    }
    float bound = n * (Pow(2, (float)1/n) - 1);
    printf ("CPU utilization: %.2f \n", utilization);
    printf ("RM Bound: %.4f \n", bound);
    if (utilization <= bound)
        printf ("Schedulable (utilization <= RM Bound) \n");
    else
        printf ("Not Schedulable \n");
    while (completed < n) {
        int min-period = 9999, selected = -1;
        for (int i=0; i<n; i++) {
            if (remaining[i] > 0 && t[i].period < min-period) {
                min-period = t[i].period;
                selected = i;
            }
        }
        if (selected != -1) {
            remaining[selected] -= 1;
            ct[selected]++;
            time += 1;
            if (ct[selected] == t[selected].period) {
                tat[selected] = time;
                completed++;
                ct[selected] = 0;
            }
        }
    }
}

```


WEEK-4

10-04-2020
26

```
#include <stdio.h>
#define MAX 10
type def struct {
    int id, int execution, int period, int deadline,
    int weight;
} Task;

void eff (Task t[], int n) {
    printf ("In earliest Deadline First (EDF) Scheduling\n");
    int time, completed = 0;
    int remaining [MAX], wt [MAX], tot [MAX];
    float utilization = 0;
    for (int i = 0; i < n; i++) {
        remaining [i] = t[i].execution;
        utilization += (float) t[i].execution / t[i].deadline;
    }
    printf ("CPU utilization: %.2f\n", utilization);
    if (utilization <= 1)
        printf ("Schedulable (utilization <= 1)\n");
    else
        printf ("Not Schedulable\n");
    while (completed < n) {
        int earliest = 9999, selected = -1;
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0 && t[i].deadline < earliest)
                earliest = t[i].deadline;
            selected = i;
        }
        if (selected != -1) {
            remaining [selected]--;
            time++;
            if (remaining [selected] == 0) {
                ct [selected] = time;
                tot [selected] = ct [selected];
                wt [selected] = tot [selected] - t[selected].period;
                completed++;
            }
        }
    }
}
```

Enter AT for P6: 6

Enter BT for P6: 3

Enter the Quantum: 6

P	AT	BT	CT	TAT	WT	RT
P1	5	5	12	4	2	2
P2	4	6	18	14	8	8
P3	3	7	23	30	23	25
P4	1	9	32	31	22	0
P5	2	2	26	24	22	22
P6	6	3	29	23	20	20

AVG WT = 16.17

AVG TAT = 21.50

AVG RT = 11.17

∴ If we increase Time quantum value, the waiting time and turnaround time gets increased but the CPU response time gets increased.

o/p

30

Enter number of process: 3

Enter process details:

Process 0: Burst Time: 3
Deadline (for EDF): 7
Period (for RMS): 20
Weight (for proportional): 5

Process 1: Burst Time: 2
Deadline (for EDF): 4
Period (for RMS): 5
Weight (for proportional): 10

Process 2: Burst Time: 2
Deadline (for EDF): 8
Period (for RMS): 10
Weight (for proportional): 8

Earliest Deadline First (EDF)

CPU Utilization = 1.18

Not Schedulable

ID	BT	Deadline	CT	WT	TAT
0	3	7	5	2	7
1	2	8	7	5	7

Rate monotonic Scheduling (RMS)

CPU utilization: 0.75

RM Bound: 0.7798

Schedulable (utilization $\leq$ RM Bound)

ID	BT	Period	CT	WT	TAT
0	3	20	7	4	7
1	2	5	2	0	2
2	1	10	4	2	4

Proportional Scheduling

Total weight: 23

ID	BT	Weight	CT	WT	TAT
0	3	5	7	4	7
1	2	10	2	0	2
2	2	8	4	2	4

```

    remaining[selected] = -1;
    ct[selected] = time;
    time++;
    if (remaining[selected] == 0) {
        ct[selected] = time;
        tat[selected] = ct[selected];
        wt[selected] = tat[selected] - t[selected].execution;
        completed++;
    }
}

```

```

}

```

```

else {
    time++;
}

```

```

printf("ID / CT / ET / weight / CT / WT / TAT \n");
for (int i = 0; i < n; i++) {
    printf("%d / %d / %d / %d / %d \n", t[i].id,
        t[i].execution, t[i].weight, ct[i], wt[i], tat[i]);
}

```

```

}

```

```

int main() {
    int n;

```

```

    Task t[MAX];

```

```

    printf("Enter number of Process:");

```

```

    scanf("%d", &n);

```

```

    printf("Enter Process details: \n");

```

```

    for (int i = 0; i < n; i++) {
        printf("Process %d: \n", i);

```

```

        t[i].id = i;
        printf("Burst Time:");

```

```

        scanf("%d", &t[i].execution);

```

```

        printf("Deadline (for RMS):");

```

```

        scanf("%d", &t[i].deadline);

```

```

        printf("Period (for RMS):");

```

```

        scanf("%d", &t[i].period);

```

```

        printf("weight (for proportional):");

```

```

        scanf("%d", &t[i].weight);
    }
}

```

```

    ct[r, n];

```

```

    rms[t, n];

```

```

    proportional(x);

```

```

    return 0;
}

```



```
remaining[selected]--;
```

```
time++;
```

```
if (remaining[selected] == 0) {
```

```
    ct[selected] = time;
```

```
    tat[selected] = ct[selected];
```

```
    wt[selected] = tat[selected] - t[selected].execution;
```

```
    completed++;
```

```
}
```

```
else {
```

```
    time++;
```

```
}
```

```
printf("ID/tBF / tPeriod / tCT/tWT/tTAT\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d\t%d\t%d\t%d\t%d\t%d\n", t[i].
```

```
    t[i].execution, t[i].period, ct[i], wt[i], tat[i]);
```

```
}
```

```
}
```

```
void Proportional (Task t[], int x) {
```

```
    printf("\n Proportional scheduling\n");
```

```
    int time = 0, completed = 0;
```

```
    int remaining[MAX], ct[MAX] = 0, wt[MAX], tat[MAX];
```

```
    int total_weight = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        remaining[i] = t[i].execution;
```

```
        total_weight += t[i].weight;
```

```
}
```

```
    printf("Total weight: %d\n", total_weight);
```

```
    while (completed < n) {
```

```
        int max_w = -1, selected = -1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (remaining[i] > 0 && t[i].weight > max_w) {
```

```
                max_w = t[i].weight;
```

```
                selected = i;
```

```
}
```

```
        if (selected != -1) {
```

Output

Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]


```

int item;
for (int i = 0; i < 10; i++)
{
    sem_wait(&full);
    pthread_mutex_lock(&mutex);
    item = buffer[out];
    printf("Consumed: %d from buffer\n",
        item, out);
    out = (out + 1) % BUFFER_SIZE;
    pthread_mutex_unlock(&mutex);
    sem_post(&empty);
    sleep(2);
}
return NULL;
}

```

```

int main()
{
    pthread_t p, c;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

LAP-5

Write a C program to simulate:

- Producer-consumer problem using semaphores.
- Dining-Philosopher's problem

```
a) → #include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;
void* producer(void* arg)
{
    int item;
    for (item = 0; item < 15; item++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d at buffer [%d]\n",
               item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
void consumer(void* arg)
```



```

for (int i=0; i<N; i++)
    sem = init (& forks[i], 0, 2);
for (int i=0; i<N; i++) {
    rds[i] = i;
    pthread = create (& thread[i],
        NULL, philosopher, &ids[i]);
}
for (int i=0; i<N; i++)
    pthread_join (thread[i], NULL);
return 0;

```

Output

Philosopher 0 is thinking.
 Philosopher 1 is thinking.
 Philosopher 2 is thinking.
 Philosopher 4 is thinking.
 Philosopher 3 is thinking.
 Philosopher 1 picked up left fork 1.
 Philosopher 1 picked up right fork 2.
 Philosopher 0 picked up left fork 0.
 Philosopher 1 is eating.
 Philosopher 1 put down forks 1 and 2.
 Philosopher 1 is thinking.
 Philosopher 0 picked up right fork 1.
 Philosopher 0 is eating.
 Philosopher 4 picked up left fork 4.
 Philosopher 4 picked up right fork 0.
 Philosopher 4 is eating.

```

by- #include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t forks[N];
sem_t mutex;
void * philosopher(void * num) {
    int id = *(int *) num;
    while(1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        sem_wait(&mutex);
        sem_wait(&forks[id]);
        printf("Philosopher %d picked up left fork %d.\n", id, id);
        sem_wait(&forks[(id+1)%N]);
        printf("Philosopher %d picked up right fork %d.\n", id, (id+1)%N);
        sem_post(&mutex);
        printf("Philosopher %d is eating.\n", id);
        sleep(2);
        sem_post(&forks[id]);
        sem_post(&forks[(id+1)%N]);
        printf("Philosopher %d put down forks %d and %d.\n", id, id, (id+1)%N);
    }
}

int main() {
    pthread_t thread[N];
    int ids[N];
    sem_init(&mutex, 0, 1);

```


Tracing

Time(s)	Active Thread	empty	full	in
0	Start	5	0	0
0	Producer	4	1	1
0	Consumer	5	0	1
1	Producer	4	1	2
2	Producer	3	2	3
2	Consumer	4	1	3
3	Producer	3	2	4
4	Producer	2	3	0
4	Consumer	3	2	0
5	Producer	2	3	1
5	Producer	1	4	2
6	Consumer	2	3	2

diff 1/5/20

```

for (int i=0; i<n; i++) finish[i]=0;
int count=0;
while (count<n) {
    int found=0;
    for (int i=0; i<n; i++) {
        if (!finish[i]) {
            int canRun=1;
            for (int j=0; j<m; j++) {
                if (need[i][j]>work[j]) {
                    canRun=0;
                    break;
                }
            }
            if (canRun) {
                for (int j=0; j<m; j++)
                    work[j] += alloc[i][j];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (!found) break;
}
printf("\n");
if (count == n) {
    printf("System is in safe state.\n");
    printf("Safe sequence is:");
    for (int i=0; i<n; i++) {
        printf("%d", safeSeq[i]);
        if (i<n-1) printf("->");
    }
}

```


- Q. Write a C program to simulate:
 (a) Banker's algorithm for the purpose of deadlock avoidance.
 (b) Deadlock Detection.

a) Banker algorithm

```
#include <stdio.h>
int main () {
    int n, m;
    printf("Enter number of processes:");
    scanf("%d", &n);
    printf("Enter number of resources:");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], need[n][m],
    avail[m];
    printf("Enter allocation matrix:\n");
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Maximum Demand Matrix:\n");
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j] - alloc[i][j];
        }
    printf("Enter Available Resources:\n");
    for (int j=0; j<m; j++)
        scanf("%d", &avail[j]);
    int finish[n], safeSeq[n];
    int work[m];
    for (int j=0; j<m; j++) work[j] = avail[j];
```

Tracing

Time (s)	Active Thread	Mutex	Final state
0	All	1	[1, 1, 1, 1, 1]
1	P0	1	[1, 1, 1, 1, 1]
1	P0	0	[0, 0, 1, 1, 1]
1	P2	1	[0, 0, 1, 1, 1]
1	P2	0	[0, 0, 0, 0, 1]
1	P1	1	[0, 0, 0, 0, 1]
1	P1	0	[0, 0, 0, 0, 1]
1	P3 & P4	0	[0, 0, 0, 0, 1]
3	P0	0	[1, 1, 0, 0, 1]
3	P1	0	[1, 0, 0, 0, 1]
3	P2	0	[1, 0, 1, 1, 1]
3	P1	0	[1, 0, 0, 1, 1]
3	P3	1	[1, 0, 0, 1, 1]

42
 for (int j=0; j<m; j++) work[j]=avail;
 int found;
 do {

found=0;
 for (int i=0; i<n; i++) {
 if (!finish[i]) {

int canRun=1;
 for (int j=0; j<m; j++) {
 if (request[i][j] > work[j]) {
 canRun=0;
 break;
 }
 }

if (canRun) {
 for (int j=0; j<m; j++)
 work[j] += alloc[i][j];
 finish[i]=1;
 found=1;
 }

}
 } while (found);
 printf("\n Processes in Deadlock:\n");
 int deadlockFound=0;
 for (int i=0; i<n; i++) {
 if (!finish[i]) {
 printf("P-%d\n", i);
 deadlockFound=1;
 }
 }

}
 if (!deadlockFound)
 printf("No deadlock detected. All processes
 can finish.\n");

b) Deadlock Detection

```
#include <stdio.h>
int main() {
    int n, m;
    printf("Enter number of Processes:");
    scanf("%d", &n);
    printf("Enter number of Resources:");
    scanf("%d", &m);
    int alloc[n][m], request[n][m], avail[m];
    printf("Enter Allocation Matrix: \n");
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Request Matrix: \n");
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++)
            scanf("%d", &request[i][j]);
    printf("Enter Available Resources: \n");
    for (int j=0; j<m; j++)
        scanf("%d", &avail[j]);
    int finish[n];
    int work[m];
    for (int i=0; i<n; i++) {
        finish[i] = 0;
        int zeroAlloc = 1;
        for (int j=0; j<m; j++) {
            if (alloc[i][j] != 0) {
                zeroAlloc = 0;
                break;
            }
        }
        if (zeroAlloc) finish[i] = 1;
    }
}
```



```

printf("\n");
} else {
printf("System is in unsafe state (Potential  
Deadlock).\n");
}
return 0;
}

```

O/P

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 2

Enter Available Resources:

3 3 2

System is in SAFE state.

Safe Sequence: P1 → P3 → P4 → P0 → P2

```
printf("\n / S fit \n", type = 0 ? "first" : type = 1;
      "Best": "Worst");
```

```
printf("PNs\tPSize\tBlock\n");
```

```
for (int i = 0; i < n; i++)
```

```
printf("%d\t%d\t%d\t%d\n", i+1, process[i],
      alloc[i] = -1 ? (char[20]) { alloc[i]+1+'0',
      0 } : "NA");
```

```
}
```

```
int main() {
```

```
int m, n;
```

```
printf("Enter no. of blocks: ");
```

```
scanf("%d", &m);
```

```
int b1[m], b2[m], b3[m], b4[m];
```

```
printf("Enter block sizes: \n");
```

```
for (int i = 0; i < m; i++)
```

```
{
```

```
scanf("%d", &b[i]);
```

```
b1[i] = b2[i] = b3[i] = b4[i];
```

```
}
```

```
printf("Enter no. of processes: ");
```

```
scanf("%d", &n);
```

```
int p[n];
```

```
printf("Enter process sizes: \n");
```

```
for (int i = 0; i < n; i++)
```

```
scanf("%d", &p[i]);
```

```
allocate(b1, m, p, n, 0);
```

```
allocate(b2, m, p, n, 1);
```

```
allocate(b3, m, p, n, 2);
```

```
return 0;
```

```
}
```


- Q. Write a C program to simulate the following contiguous memory allocation techniques:
- Worst-fit
 - Best-fit
 - First-fit

```
#include <stdio.h>
void allocate(int blocks[], int m, int process[], int n,
              int type)
```

```
{
    int alloc[n];
    for (int i = 0; i < n; i++) alloc[i] = -1;
    for (int i = 0; i < n; i++)
```

```
{
    int idx = -1;
    for (int j = 0; j < m; j++)
```

```
{
    if (blocks[j] >= process[i])
```

```
{
    if (type == 0) { idx = j; break; }
    if (idx == -1 ||
```

```
(type == 1 && blocks[j] < blocks[idx])
```

```
|| (type == 2 && blocks[j] > blocks[idx])
```

```
idx = j;
```

```
}
```

```
if (idx != -1)
```

```
alloc[i] = idx;
```

```
blocks[idx] -= process[i];
```

```
}
```

```
}
```

return 0;

3

O/P

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 1

0 0 2

Enter Available Resources:

0 0 0

Processes in Deadlock:

No deadlock detected: All processes can finish.

07/05/26

Write a C program to simulate page replacement algorithms.

- FIFO
- LRU
- Optimal

```
→ #include <stdio.h>
void printFrames(int frames[], int f) {
    int i;
    for (i = 0; i < f; i++)
        (frames[i] == -1) ? printf("-") : printf("%d", frames[i]);
    printf("\n");
}

void FIFO (int p[], int n, int f) {
    int fr[f], i, j, k = 0, found, faults = 0;
    for (i = 0; i < f; i++) fr[i] = -1;
    printf("\n--- FIFO Page Replacement ---\n");
    printf("Page\tFrames\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++)
            if (fr[j] == p[i]) found = 1;
        if (!found) {
            fr[k] = p[i];
            k = (k + 1) % f;
            faults++;
            printf("%d\t", p[i]);
            printFrames(fr, f);
        }
    }
    printf("\n Total FIFO Page Faults = %d", faults);
}
```

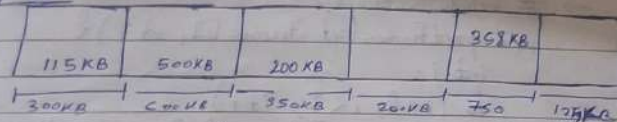
Memory Partition

300KB, 600KB, 350KB, 200KB, 750KB, 125KB (in order)

Process Size

115KB, 500KB, 350KB, 200KB, 375KB (in order)

First Fit

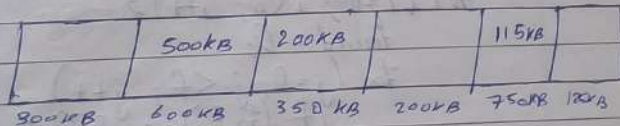


process with size 375KB cannot be allocated with memory.

Best Fit



Worst Fit



Process size 375KB & 375KB cannot be allocated with memory.

Sol

Output

Enter block sizes:

100 500 200 300 600

Enter no. of processes: 4

Enter process sizes:

212 417 112 426

First Fit

PNo	Psize	Block
1	212	2
2	417	5
3	112	2
4	426	NA

Best Fit

PNO	Psize	Block
1	212	4
2	417	2
3	112	3
4	426	5

Worst Fit

PNo	Psize	Block
1	212	5
2	417	2
3	112	5
4	426	NA

```

    p[pos] = p[i];
    faults++;
    printf("%d\t", p[i]);
    printf("\n");
}
}

printf("\n Total Optimal Page Faults = %d\n",
    faults);
}

int main() {
    int pages[50], n, f, i;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string: \n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    FIFO(pages, n, f);
    LRU(pages, n, f);
    Optimal(pages, n, f);
    return 0;
}

```

O/R Enter number of pages: 12
 Enter page reference string:
 1 2 3 4 1 2 5 1 2 3 4 5
 Enter number of frames: 3


```
printf("\n--- Optical Page Requirement ---\n");
printf("Page \t\t Names\n");
for (i = 0; i < n; i++) {
```

Printed ("Page \ t. Francis \")

```
for (i = 0; i < n; i++) {
```

$$I_{\text{quad}} = 0,$$

```
for (j = 0; j < 7; j++)
```

```

if ( ! found || p[i] == p[j] ) found =
if (! found) {

```

$$i \neq (! \text{ found}) \frac{1}{2}$$
$$p_{05} = -1;$$
$$f_{\text{for}}(j=0, j < 7, j++)$$
$$i \in (L_j) = -1$$
$$p_{os} = j$$

break

4

$$i\hbar(\rho_{00} = -1) \cdot \xi$$
$$f_{\text{arr}}[i] = i + 1;$$
$$p_{OS} = 0;$$
$$\text{for } (j=0; j \leq 7; j++)$$
$$\text{index} = -1$$

for ($k=i+1$; $k \leq n$; $k++$)

$$i \neq j \text{ (} \text{tr}[j] = \text{pr}[j] \text{)}$$

index = k

break;

2

if (index == -1) {

pos = 1

break,

2

if (index > farthest)

2. $\text{Pos} = \text{index}$

$$Pos = j;$$

4

4

```

void LRU (int p[], int n, int f) {
    int fr [20], t [20] = {0};
    int i, j, found, faults = 0, cnt = 0, least;
    for (i = 0; i < f; i++) fr[i] = -1;
    printf("\n --- LRU Page Replacement --- \n");
    printf("Page \t Frames \n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++) {
            if (fr[j] == p[i]) {
                t[j] = ++cnt;
                found = 1;
            }
        }
        if (!found) {
            least = 0;
            for (j = 1; j < f; j++) {
                if (t[j] < t[least]) least = j;
            }
            fr[least] = p[i];
            t[least] = ++cnt;
            faults++;
            printf("/d \t", p[i]);
            printfFrames(fr, f);
        }
    }
    printf("\n Total LRU Pages Faults = /d \n", faults);
}

```

```

void Optimal (int p[], int n, int f) {
    int fr [20], i, j, k, found, faults = 0;
    int pos, farthest, index;
    for (i = 0; i < f; i++) fr[i] = -1;
}

```


--- FIFO Page Replacement ---

Page	Frames
1	1 - -
2	1 2 -
3	1 2 3
4	4 2 3
1	4 1 3
2	4 1 2
5	5 1 2
3	5 3 2
4	5 3 4

Total FIFO Page Faults = 9

--- LRU Page Replacement ---

Pages	Frames
1	1 - -
2	1 2 -
3	1 2 3
4	4 2 3
1	4 1 3
2	4 1 2
5	5 1 2
3	3 1 2
4	3 4 2
5	3 4 5

Total LRU Page Faults = 10

--- Optimal Page Replacement ---

Pages	Frames
1	1 - -
2	1 2 -
3	1 2 3
4	1 2 4
5	1 2 5
3	3 2 5
4	4 2 5

Total Optimal Page Faults = 7

~~Q.8~~
~~2/15/26~~